

Robo-Marketplaces — melhorias para o negócio de esmaltes Impala

Análise do código + 6 módulos novos baseados na inteligência de mercado construída nesta sessão

O robô está bem estruturado — repricing, chat IA, NF-e e monitoramento já existem. O que falta são contextos específicos do negócio de esmaltes que só você tem após essa sessão de inteligência de mercado.

8

MÓDULOS JÁ PRONTOS

6

MÓDULOS A CRIAR

2

CONFIGS A AJUSTAR

1

CATÁLOGO A MONTAR

O que o robô já faz bem

Repricing automático

Chat IA

NF-e Bling

Saúde da conta ML

Meta Ads monitor

Telegram alertas

Operação 24h

Keepalive contas

O que está faltando para esmaltes

Catálogo real dos kits

Margem por fase (1/2/3)

Gatilho de ads por avaliações

Alerta frete vs margem

Controle de fases

Sazonalidade (Dia das Mães etc.)

Problema crítico no catálogo atual

O arquivo `catalogo/produtos.json` atual tem apenas 2 produtos fictícios (ESM-001 e ESM-002). Para o robô funcionar de verdade ele precisa dos 7 kits reais com custos exatos, margens por fase e canais corretos. Sem isso o repricing e o monitoramento de margem funcionam com dados errados.

Configurações da spec.yaml que precisam ajuste

A `margem_minima_pct: 15.0` na spec não reflete a realidade — na Fase 1 você opera com 10-12% para ganhar avaliações, e na Fase 3 com Full você pode exigir 25%+. O robô precisa saber em qual fase está para não bloquear preços competitivos da Fase 1 achando que é prejuízo.

Substitua o arquivo `catalogo/produtos.json` pelo JSON abaixo. Ele tem os 7 kits com custos reais, preços por fase, caixas CaixasNet e frete estimado de Americana/SP.

```
// catalogo/produtos.json - Kits Esmaltes Impala (real)
[
  {
    "sku": "IMP-MIMO-003",
    "nome": "Kit 3 Mimo + Carmed Impala",
    "ncm": "3304.10.00",
    "custo_esmaltes": 8.61,
    "custo_complemento": 7.00,
    "custo_embalagem": 1.56,
    "custo_caixa": 0.98,
    "custo_total_sem_frete": 18.15,
    "frete_estimado": 9.00,
    "custo_total": 27.15,
    "fase_atual": 1,
    "precos_por_fase": {
      "fase1": 44.90,
      "fase2": 49.90,
      "fase3": 54.90
    },
    "preco": 44.90,
    "estoque_total": 0,
    "peso_gramas": 350,
    "caixa_caixasnet": "S20-16x11x10",
    "canais": {
      "mercadolivre": {"ativo": true, "preco": 44.90, "estoque": 0},
      "shopee": {"ativo": false, "preco": 0, "estoque": 0}
    }
  },
  {
    "sku": "IMP-BAIL-005",
    "nome": "Kit 5 Bailarina Impala",
    "ncm": "3304.10.00",
    "custo_esmaltes": 14.35,
    "custo_complemento": 5.00,
    "custo_embalagem": 3.90,
    "custo_caixa": 0.98,
    "custo_total_sem_frete": 24.23,
    "frete_estimado": 11.00,
    "custo_total": 35.23,
    "fase_atual": 1,
    "precos_por_fase": {
      "fase1": 48.90,
      "fase2": 52.90,
      "fase3": 57.90
    },
    "preco": 48.90,
    "estoque_total": 0,
    "peso_gramas": 530,
    "caixa_caixasnet": "S20-16x11x10",
    "canais": {
      "mercadolivre": {"ativo": true, "preco": 48.90, "estoque": 0},
      "shopee": {"ativo": false, "preco": 0, "estoque": 0}
    }
  }
],
```

```
{
  "sku": "IMP-SORT-006",
  "nome": "Kit 6 Sortido Cores da Moda Impala",
  "ncm": "3304.10.00",
  "custo_esmaltes": 17.22,
  "custo_complemento": 1.00,
  "custo_embalagem": 4.18,
  "custo_caixa": 3.70,
  "custo_total_sem_frete": 26.10,
  "frete_estimado": 13.00,
  "custo_total": 39.10,
  "fase_atual": 1,
  "precos_por_fase": {
    "fase1": 49.90,
    "fase2": 54.90,
    "fase3": 59.90
  },
  "preco": 49.90,
  "estoque_total": 0,
  "peso_gramas": 620,
  "caixa_caixasnet": "S03-31x20x11.5",
  "canais": {
    "mercadolivre": {"ativo": true, "preco": 49.90, "estoque": 0},
    "shopee": {"ativo": false, "preco": 0, "estoque": 0}
  }
},
{
  "sku": "IMP-ATAC-010",
  "nome": "Kit 10 Atacado Manicure Impala",
  "ncm": "3304.10.00",
  "custo_esmaltes": 28.70,
  "custo_complemento": 1.00,
  "custo_embalagem": 5.04,
  "custo_caixa": 3.70,
  "custo_total_sem_frete": 38.44,
  "frete_estimado": 14.50,
  "custo_total": 52.94,
  "fase_atual": 1,
  "precos_por_fase": {
    "fase1": 69.90,
    "fase2": 74.90,
    "fase3": 79.90
  },
  "preco": 69.90,
  "estoque_total": 0,
  "peso_gramas": 1000,
  "caixa_caixasnet": "S03-31x20x11.5",
  "canais": {
    "mercadolivre": {"ativo": true, "preco": 69.90, "estoque": 0},
    "shopee": {"ativo": false, "preco": 0, "estoque": 0}
  }
}
}
```

```
]
```

O repricing atual usa margem mínima fixa. Para esmaltes isso não funciona — na Fase 1 você aceita 10% para ganhar avaliações. O robô precisa saber a fase atual de cada kit para precificar corretamente.

Novo módulo: agente_repricing_impala.py

Extensão do repricing existente que respeita as 3 fases. Adicione este arquivo em `agentes/repricing/`
`agente_repricing_impala.py` :

```

# agentes/repricing/agente_repricing_impala.py
# Repricing consciente de fase para kits Impala

import json
from pathlib import Path

TAXA_ML = 0.14
CATALOGOS_PATH = Path("catalogo/produtos.json")

MARGEM_POR_FASE = {
    1: 0.10, # Fase 1: 10% - ganhar avaliações
    2: 0.18, # Fase 2: 18% - crescimento
    3: 0.25, # Fase 3: 25% - reputação/Full
}

def carregar_kits() -> list:
    with open(CATALOGOS_PATH) as f:
        return json.load(f)

def calcular_preco_ideal(kit: dict, fase: int = None) -> dict:
    fase = fase or kit.get("fase_atual", 1)
    custo = kit["custo_total"]
    margem_min = MARGEM_POR_FASE.get(fase, 0.10)

    # Preço mínimo para não dar prejuízo
    preco_min_lucro = custo / (1 - TAXA_ML - margem_min)

    # Preço alvo da fase
    preco_fase = kit.get("precos_por_fase", {}).get(f"fase{fase}", preco_min_lucro)

    # Nunca abaixo do mínimo de lucro
    preco_sugerido = max(preco_fase, preco_min_lucro)

    lucro = preco_sugerido * (1 - TAXA_ML) - custo
    margem_real = lucro / preco_sugerido

    return {
        "sku": kit["sku"],
        "nome": kit["nome"],
        "fase": fase,
        "custo_total": custo,
        "preco_atual": kit["preco"],
        "preco_sugerido": round(preco_sugerido, 2),
        "lucro_estimado": round(lucro, 2),
        "margem_real_pct": round(margem_real * 100, 1),
        "ajuste_necessario": abs(preco_sugerido - kit["preco"]) >= 0.50,
    }

def executar(dry_run: bool = True) -> dict:
    kits = carregar_kits()
    resultados = [calcular_preco_ideal(k) for k in kits]
    ajustes = [r for r in resultados if r["ajuste_necessario"]]
    return {"dry_run": dry_run, "total": len(resultados), "ajustes": ajustes, "detalhes": resultados}

```

O robô monitora saúde da conta mas não sabe quando ligar ou desligar os ads. Adicione este agente que toma a decisão baseada em avaliações, reputação e ACOS real.

Novo módulo: agente_ads_gatilho.py

Adicione em `agentes/ml/agente_ads_gatilho.py`. Ele consulta as avaliações reais da conta e decide automaticamente se é hora de ligar o Product Ads, escalar ou pausar.

```

# agentes/ml/agente_ads_gatilho.py
# Decide automaticamente quando ligar/escalar/pausar Product Ads no ML

from core.notificador import alertar_gestor
from integracoes.ml.ml_client import buscar_reputacao_vendedor

# Thresholds baseados na análise dos top sellers
AVALIACOES_PARA_ADS      = 20    # mínimo de avaliações para ligar ads
NOTA_MINIMA_PARA_ADS     = 4.8   # nota mínima para ligar ads
AVALIACOES_PARA_ESCALAR  = 50    # mínimo para escalar budget
ACOS_MAXIMO_ACEITAVEL    = 0.20  # 20% – se ACOS > isso, pausar
BUDGET_FASE_INICIO       = 10.0   # R$/dia fase inicial
BUDGET_FASE_CRESCIMENTO  = 30.0   # R$/dia com Full
BUDGET_FASE_ESCALA       = 80.0   # R$/dia pico sazonal

def avaliar_momento_ads(avaliacoes: int, nota_media: float, acos_atual: float = 0.0, full_ativo: bool = False):
    decisao = "aguardar"
    budget_sugerido = 0.0
    motivos = []

    if avaliacoes < AVALIACOES_PARA_ADS:
        motivos.append(f"Avaliações insuficientes: {avaliacoes}/{AVALIACOES_PARA_ADS}")
        motivos.append("Focar em orgânico + Programa Decola")

    elif nota_media < NOTA_MINIMA_PARA_ADS:
        motivos.append(f"Nota abaixo do mínimo: {nota_media:.1f}/{NOTA_MINIMA_PARA_ADS}")
        motivos.append("Melhorar atendimento antes de investir em ads")

    elif acos_atual > ACOS_MAXIMO_ACEITAVEL and acos_atual > 0:
        decisao = "pausar"
        budget_sugerido = 0.0
        motivos.append(f"ACOS alto: {acos_atual*100:.0f}% (máx {ACOS_MAXIMO_ACEITAVEL*100:.0f}%)")
        motivos.append("Revisar título e preço antes de religar")

    elif full_ativo and avaliacoes >= AVALIACOES_PARA_ESCALAR:
        decisao = "escalar"
        budget_sugerido = BUDGET_FASE_CRESCIMENTO
        motivos.append("Full ativo + volume sólido = momento de escalar")

    elif avaliacoes >= AVALIACOES_PARA_ADS:
        decisao = "ligar"
        budget_sugerido = BUDGET_FASE_INICIO
        motivos.append("Avaliações suficientes para iniciar Product Ads")
        motivos.append(f"Budget sugerido: R$ {BUDGET_FASE_INICIO}/dia · Campanha automática")

    resultado = {
        "decisao": decisao,
        "budget_sugerido_dia": budget_sugerido,
        "avaliacoes": avaliacoes,
        "nota_media": nota_media,
        "acos_atual": acos_atual,
        "full_ativo": full_ativo,
        "motivos": motivos,
    }

```

```

if decisao == "ligar":
    alertar_gestor(f"ADS: momento de LIGAR Product Ads\nBudget: R$ {budget_sugerido}/dia\n{''; '.join(motivos)}")
elif decisao == "pausar":
    alertar_gestor(f"ADS: PAUSAR – ACOS alto {acos_atual*100:.0f}%\n{''; '.join(motivos)}")

return resultado

def executar(acos_atual: float = 0.0, full_ativo: bool = False) -> dict:
    rep = buscar_reputacao_vendedor()
    metrics = rep.get("metrics", {})
    avaliacoes = int(metrics.get("total_ratings", 0))
    nota = float(metrics.get("average_rating", 0.0))
    return avaliar_momento_ads(avaliacoes, nota, acos_atual, full_ativo)

```

5 alertas específicos para esmaltes a adicionar no notificador

- 1 **Alerta de frete crítico:** quando custo de frete supera 35% do custo total do kit, o robô avisa para revisar a faixa de peso ou considerar ML Full. Adicione em agentes/ml/agente_ml.py dentro do ciclo_chat().
- 2 **Alerta de avaliações:** quando atingir 20, 50 e 100 avaliações, o Telegram avisa com a ação recomendada (ligar ads, subir preço de fase, lançar kit maior).
- 3 **Alerta sazonal:** 30 dias antes do Dia das Mães, Natal e Dia dos Namorados, o robô sugere aumentar estoque dos kits mimo e gift e lançar campanha de ads sazonal.
- 4 **Alerta de concorrente:** quando o repricing detectar que um concorrente baixou preço mais de 10% nos kits 6 e 10, avisa imediatamente para avaliar resposta — não repricia automaticamente abaixo da margem mínima da fase.
- 5 **Alerta de estoque crítico:** o robô já tem este alerta (ESTOQUE_CRITICO=20 na spec), mas ajuste para 30 unidades nos kits de validação (3, 5, 6) que precisam de giro mais rápido.

```

# Adicionar em core/notificador.py ou criar alertas/esmaltes.py

from datetime import datetime
from core.notificador import alertar_gestor

DATAS_SAZONAIAS = [
    ("05-11", "Dia das Mães"),    # 2º domingo de maio
    ("06-12", "Dia dos Namorados"),
    ("12-10", "Natal – antecipação"),
]

MARCOS_AVALIACOES = {
    20: "Ligar Product Ads R$10/dia · campanha automática",
    50: "Subir preços para Fase 2 nos kits com 4.8+",
    100: "Lançar kit 10 atacado · avaliar ancoragem de preço",
    230: "MercadoLíder desbloqueado – aumentar budget ads",
}

def verificar_marcos_avaliacoes(total_avaliacoes: int):
    for marco, acao in MARCOS_AVALIACOES.items():
        if total_avaliacoes >= marco:
            alertar_gestor(f"Marco atingido: {total_avaliacoes} avaliações!\nAção: {acao}")
            break

def verificar_sazonalidade():
    hoje = datetime.now().strftime("%m-%d")
    for data, evento in DATAS_SAZONAIAS:
        mes_dia = datetime.strptime(data, "%m-%d")
        diff = (mes_dia - datetime.strptime(hoje, "%m-%d")).days
        if 25 <= diff <= 35:
            alertar_gestor(
                f"Sazonalidade: {evento} em ~{diff} dias\n"
                "Ações: aumentar estoque kits mimo, lançar campanha ads sazonal, "
                "criar bundle presente com Carmed"
            )

```

Substitua o conteúdo de spec/spec.yaml por este novo spec que reflete o negócio real de esmaltes Impala com as 3 fases de operação.

```
# spec/spec.yaml - Robo-Marketplaces Esmaltes Impala
projeto:
  nome: Robo-Markplaces-Impala
  versao: "2.0"
  descricao: >
    Agente autônomo para venda de kits de esmaltes Impala no ML.
    Opera em 3 fases: validação (0-20 avaliações), crescimento
    (20-100 avaliações) e reputação/Full (100+ avaliações).

marketplaces:
  - id: mercadolibre
    ativo: true
    prioridade: 1
  - id: shopee
    ativo: false
    prioridade: 2

fase_operacao:
  fase_atual: 1 # Altere conforme evolui: 1, 2 ou 3
  full_ativo: false
  mercadolider: false

regras_negocio:
  # Margem mínima por fase
  margem_minima_fase1_pct: 10.0 # aceita menor para ganhar avaliações
  margem_minima_fase2_pct: 18.0 # crescimento
  margem_minima_fase3_pct: 25.0 # reputação/Full
  taxa_marketplace_pct: 14.0 # taxa ML categoria beleza

  # Estoque
  estoque_critico_unidades: 30 # ajustado para kits de validação
  estoque_ideal_unidades: 60

  # Ads (Product Ads ML)
  ads_avaliacoes_minimas: 20 # não liga ads antes disso
  ads_nota_minima: 4.8
  ads_budget_inicio_dia: 10.0 # R$/dia fase 2
  ads_budget_full_dia: 30.0 # R$/dia fase 3
  ads_acos_maximo_pct: 20.0 # pausa se ACOS > 20%

  # Meta Ads
  cpc_maximo_reais: 1.50
  ctr_minimo_pct: 1.20 # ajustado para manicure
  roas_escala: 2.20 # ajustado para manicure

  # Repricing
  repricing_variacao_minima: 0.50 # só ajusta se diferença >= R$0,50
  repricing_nunca_abaixo_custo: true

  # Horários
  horario_post_manha: "08:00"
  horario_post_noite: "19:00"
  varredura_hora: 6
  varredura_minuto: 0
```

```
kits_prioritarios:
  validacao: ["IMP-MIMO-003", "IMP-SORT-006"] # lançar primeiro
  fase2: ["IMP-BAIL-005", "IMP-ATAC-010"]
  fase3: ["IMP-COLC-015", "IMP-GRAN-030"]

datas_sazonais:
  - data: "05-11"
    evento: "Dia das Mães"
    dias_antecedencia_alerta: 30
  - data: "06-12"
    evento: "Dia dos Namorados"
    dias_antecedencia_alerta: 21
  - data: "12-10"
    evento: "Natal"
    dias_antecedencia_alerta: 45
```

Resumo dos 6 arquivos a criar/modificar para integrar a inteligência desta sessão no robô.

1 catalogo/produtos.json — substituir pelos 7 kits reais

Prioridade máxima

Ver aba "Catálogo de kits" para o JSON completo com SKUs, custos reais, preços por fase e configurações de caixa. Sem isso, repricing e margem estão calculando com dados fictícios.

2 spec/spec.yaml — atualizar regras de negócio

Prioridade máxima

Ver aba "Spec atualizado". Inclui as 3 fases de margem, thresholds de ads corretos (20 avaliações mínimas), datas sazonais e lista de kits prioritários por fase.

3 agentes/repricing/agente_repricing_impala.py — criar

Alta prioridade

Ver aba "Repricing por fase". Estende o repricing existente com consciência de fase — Fase 1 aceita 10%, Fase 3 exige 25%. Evita que o robô bloqueie preços competitivos achando que é prejuízo.

4 agentes/ml/agente_ads_gatilho.py — criar

Alta prioridade

Ver aba "Gatilho de ads". Monitora avaliações e nota média da conta ML e decide automaticamente quando ligar, escalar ou pausar o Product Ads. Envia alerta no Telegram com o budget sugerido.

5 core/alertas_esmaltes.py — criar

Média prioridade

Ver aba "Alertas inteligentes". Alertas de marcos de avaliações, sazonalidade (Dia das Mães, Natal), frete crítico e concorrência. Integrar no workflow de operação 24h existente (`agentes/operacao_24h.py`).

O agente de tráfego atual classifica campanhas em `impala`, `anita`, `kits` e `outras`. Adicione o grupo `cruzeiro` no dicionário `_BRAND_KEYS` em `agente_trafego_manicures.py` para monitorar as campanhas do kit esmalte + removedor Cruzeiro separadamente.

```
_BRAND_KEYS = {
    "impala": ["impala"],
    "anita": ["anita"],
    "cruzeiro": ["cruzeiro", "removedor", "acetona"], # NOVO
    "kits": ["kit", "kits", "combo", "manicure kit"],
}
```

Ordem de implementação recomendada:

1. Atualizar `catalogo/produtos.json` com os 7 kits reais
2. Atualizar `spec/spec.yaml` com as regras de negócio corretas
3. Criar `agente_repricing_impala.py` e integrar no endpoint `/repricing`
4. Criar `agente_ads_gatilho.py` e chamar na varredura diária
5. Criar `alertas_esmaltes.py` e integrar no `operacao_24h.py`
6. Adicionar grupo `cruzeiro` no agente de tráfego de manicures